PetriNetViewer コンポーネント UI仕様書

1. 概要

1.1 コンポーネントの目的

PetriNetViewerは、MAIMLファイルからPetri Net図を生成し、React Flowを使用してインタラクティブに表示するビューアコンポーネントです。ノードクリックによるプロパティ表示、サブネット表示、エクスポート機能を提供します。

1.2 使用場面

- プロジェクト管理画面でMAIMLファイルを選択した際の表示
- 進捗確認画面で実験フローを可視化する際
- フローティングダイアログ内でのPetri Net表示
- 複数メソッドを含むMAIMLファイルの個別メソッド表示

1.3 アクセス権限

- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: JWT + Session Hybrid認証

---

2. 画面構成

2.1 レイアウト構造

2.1.1 通常表示時

```
┌──────────────────────────────────────────────────────┐
│ ペトリネット図  Enhanced Mode  📋 [メソッド名]          │
│ ノード: 12 | エッジ: 15  [アーク名表示 Switch] [エクスポート] │
├──────────────────────────────────────────────────────┤
│                                                      │
│                React Flow Canvas                     │
│                                                      │
│  ○ Material      ⬡ Transition      ○ Result          │
│  ○ Condition                                         │
```

```
│                                                          │
│  [MiniMap]                          [Controls]           │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

2.1.2 ロード中

```
┌──────────────────────────────────────────────────────────┐
│                                                          │
│                    🔄 Spinner                            │
│                ペトリネット構造を読み込み中...             │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

2.1.3 エラー時

```
┌──────────────────────────────────────────────────────────┐
│  ⚠️ エラー                                               │
│     [エラーメッセージ]                                    │
└──────────────────────────────────────────────────────────┘
```

2.1.4 プロパティDrawer表示時

```
┌──────────────────────────────────────┬──────────────────┐
│                                      │ Property Drawer  │
│         React Flow Canvas            │                  │
│                                      │ [Basic]          │
│                                      │ [Template]       │
│                                      │ [Data]           │
│                                      │                  │
│                                      │ [選択プロパティ]  │
│                                      │                  │
└──────────────────────────────────────┴──────────────────┘
```

2.2 主要UI要素

2.2.1 ヘッダー

- **左側**:
  - タイトル: "ペトリネット図 (React Flow)"（Title level 5）
  - Enhanced Mode表示（条件付き）
  - メソッド名表示（複数メソッド対応時）

    - ノード数・エッジ数

- **右側**:
    - アーク名表示Switch
    - エクスポートドロップダウンボタン

2.2.2 React Flow Canvas

- **カスタムノードタイプ**:
    - `materialTemplate`: MaterialTemplateNode（Material Place）
    - `conditionTemplate`: ConditionTemplateNode（Condition Place）
    - `resultTemplate`: ResultTemplateNode（Result Place）
    - `transition`: TransitionNode（Transition）

- **Background**: グリッドパターン
- **Controls**: ズーム、フィット、ロック等のコントロール
- **MiniMap**: 120x200px、ズーム・パン可能

2.2.3 プロパティDrawer

- **幅**: 450px
- **コンテンツ**: PropertyViewerWithSelectionコンポーネント
- **構造化プロパティ**:
    - Basic: ID、Type、Name、Description
    - Template: テンプレートプロパティ
    - Data: 座標等のデータプロパティ

---

3. 機能仕様

3.1 MAIMLデータの解析と変換

3.1.1 処理フロー

```
useEffect(() => {
  if (!maimlData) {
    setNodes([]);
    setEdges([]);
    setError(null);
    return;
  }

  setLoading(true);
  setError(null);

  try {
    // 新方式: MaimlParserTyped でパース
    const parser = new MaimlParserTyped(maimlData);
    const methodsData: Method[] = parser.getMethods();

    // selectedMethodId が指定されている場合は対象メソッドのみ
    const targetMethod = selectedMethodId
      ? methodsData.find((m: Method) => m.id === selectedMethodId) || methodsData[0]
      : methodsData[0];

    if (!targetMethod) {
      setNodes([]);
      setEdges([]);
      setError('メソッドが見つかりません');
      return;
    }

    // templateMap を構築
    const builtTemplateMap = buildTemplateMap(maimlData);

    // SubNetViewer 用の構造データを生成
    const placesForSubNet = (targetMethod.pnml.places || []).map((place) => {
      const tmpl = builtTemplateMap[place.id];
      return {
        id: place.id,
        data: {
          label: place.name || place.id,
          stereotype: tmpl?.category || 'material',
          template: tmpl ? { id: tmpl.id } : undefined,
          templateId: tmpl?.id,
        },
      };
    });

    setPetriNetStructure({
      places: placesForSubNet,
      arcs: targetMethod.pnml.arcs || [],
    });

    // templateMap を SubNetViewer 用の { element: Element } 形式に変換
    const xmlDoc = new DOMParser().parseFromString(maimlData, 'text/xml');
    const newTemplateMap: Record<string, { element: Element }> = {};
    Object.entries(builtTemplateMap).forEach(([placeId, tmpl]) => {
```

```
      const el = xmlDoc.querySelector(
        `materialTemplate[id="${tmpl.id}"], conditionTemplate[id="${tmpl.id}"],
resultTemplate[id="${tmpl.id}"]`
      );
      if (el) {
        newTemplateMap[placeId] = { element: el };
      }
    });
    setTemplateMap(newTemplateMap);

    // React Flow 形式に変換
    const petriNetData = {
      methods: [targetMethod],
      places: targetMethod.pnml.places || [],
      transitions: targetMethod.pnml.transitions || [],
      arcs: targetMethod.pnml.arcs || [],
    };
    const { nodes: flowNodes, edges: flowEdges } = convertToReactFlowFormat(
      parser,
      petriNetData,
      builtTemplateMap
    );

    setNodes(flowNodes);
    setEdges(flowEdges);
  } catch (err) {
    console.error('❌ [PetriNetViewer] 初期化エラー:', err);
    setError(err instanceof Error ? err.message : 'ペトリネット構造の読み込みに失敗しました');
  } finally {
    setLoading(false);
  }
}, [maimlData, selectedMethodId, setNodes, setEdges]);
```

3.1.2 UI動作

- **開始時**: loadingフラグをtrueに設定、Spinner表示
- **パース**: MaimlParserTypedでMAIMLをパース
- **メソッド選択**: selectedMethodIdが指定されている場合は該当メソッドのみ、なければ最初のメソッド
- **変換**: convertToReactFlowFormatでReact Flow形式に変換
- **成功時**: nodes、edgesをStateに設定、Canvas表示
- **エラー時**: errorメッセージを設定、Alertで表示

3.2 ノードクリック処理

3.2.1 Place（Material/Condition/Result）ノードクリック

```
const handleNodeClick = useCallback((event: React.MouseEvent, node: Node) => {
  setSelectedNodeId(node.id);
  setSelectedNode(node);

  if (node.data?.nodeType === 'place') {
    // 基本情報
    const basicProperties: Record<string, PropertyObject> = {
      id: { key: 'id', value: String(node.id), type: 'string', description: 'ノードID' },
      type: { key: 'type', value: String(node.data.templateType || 'material'), type: 'string',
description: 'ノード種別' },
      name: { key: 'name', value: String(node.data.label || node.id), type: 'string', description: 'ノー
ド名' },
      description: { key: 'description', value: String(node.data.description || '説明なし'), type:
'string', description: 'ノード説明' }
    };

    // テンプレートプロパティ
    const templateProperties: Record<string, PropertyObject> = {};
    if (node.data.properties && typeof node.data.properties === 'object') {
      Object.entries(node.data.properties).forEach(([key, value]) => {
        templateProperties[key] = {
          key,
          value: value || '値なし',
          type: 'string',
          description: '説明なし'
        };
      });
    }

    // データプロパティ
    const dataProperties: Record<string, PropertyObject> = {
      coordinates: {
        key: 'coordinates',
        value: `(${node.position.x.toFixed(2)}, ${node.position.y.toFixed(2)})`,
        type: 'coordinates',
        description: 'ノード座標'
      }
    };

    setStructuredProperties({
      basic: basicProperties,
      template: templateProperties,
      data: dataProperties
    });

    setDrawerOpen(true);
  }
}, []);
```

3.2.2 Transitionノードクリック

```
if (node.data?.nodeType === 'transition') {
  if (!petriNetStructure || !maimlData) {
    message.error('サブネット情報が取得できません');
    return;
  }

  if (onOpenSubNetViewer) {
    onOpenSubNetViewer({
      transitionName: node.id,
      maimlData,
      templateMap,
      petriNetData: petriNetStructure,
    });
  } else {
    message.info('サブネット表示機能は準備中です');
  }
}
```

- **Placeノード**: プロパティDrawerを開く
- **Transitionノード**: SubNetViewerダイアログを開く（親コンポーネントに委譲）

3.3 プロパティ選択機能

3.3.1 プロパティトグル

```
const handlePropertyToggle = useCallback((structure: 'basic' | 'template' | 'data', property: string) =>
{
  if (!selectedNodeId) return;

  setSelectedPropertiesByNode(prev => {
    const nodeProps = prev[selectedNodeId] || { basic: [], template: [], data: [] };
    const newNodeProps = { ...nodeProps };
    const currentList = newNodeProps[structure] || [];

    if (currentList.includes(property)) {
      newNodeProps[structure] = currentList.filter(p => p !== property);
    } else {
      newNodeProps[structure] = [...currentList, property];
    }

    const newState = {
      ...prev,
      [selectedNodeId]: newNodeProps
    };

    // 選択されたプロパティをノードデータに反映
    setNodes(nds => nds.map(node => {
```

```
      if (node.id === selectedNodeId) {
        const selectedPropsData: Record<string, string> = {};

        Object.entries(newNodeProps).forEach(([struct, props]) => {
          props.forEach(propKey => {
            const propData = structuredProperties[struct as keyof typeof structuredProperties]?.
[propKey];
            if (propData && typeof propData === 'object' && 'value' in propData) {
              selectedPropsData[propKey] = String(propData.value);
            }
          });
        });

        return {
          ...node,
          data: {
            ...node.data,
            selectedProperties: selectedPropsData,
          }
        };
      }
      return node;
    }));

    return newState;
  });
}, [selectedNodeId, structuredProperties, setNodes]);
```

- **機能**: プロパティのチェックボックスをトグル
- **効果**: 選択されたプロパティがノードデータの `selectedProperties` に反映される

3.4 アーク名表示

3.4.1 処理フロー

```
const displayEdges: Edge[] = useMemo(() => {
  return edges.map(edge => ({
    ...edge,
    label: showArcLabels ? (edge.data?.name || edge.label) : undefined,
    labelStyle: showArcLabels ? { fill: '#666', fontWeight: 400, fontSize: 10 } : undefined,
    labelBgStyle: showArcLabels ? { fill: '#f5f5f5', fillOpacity: 0.9 } : undefined,
    labelBgPadding: showArcLabels ? [4, 2] as [number, number] : undefined,
    labelBgBorderRadius: showArcLabels ? 3 : undefined,
  }));
}, [edges, showArcLabels]);
```

- **Switch制御**: `showArcLabels` Stateで切り替え

- **ラベル表示**: edgeのnameまたはlabelを表示
- **スタイル**: グレー文字、白背景、角丸

3.5 エクスポート機能

3.5.1 JSON形式

```
{
  key: 'json',
  label: 'JSON形式として保存',
  onClick: () => {
    const flowData = {
      nodes,
      edges,
      metadata: {
        exportedAt: new Date().toISOString(),
        nodeCount: nodes.length,
        edgeCount: edges.length,
      }
    };

    const dataStr = JSON.stringify(flowData, null, 2);
    const dataBlob = new Blob([dataStr], { type: 'application/json' });
    const url = URL.createObjectURL(dataBlob);

    const link = document.createElement('a');
    link.download = `petri-net-${new Date().getTime()}.json`;
    link.href = url;
    link.click();

    URL.revokeObjectURL(url);
    message.success('JSON形式で保存しました');
  },
}
```

3.5.2 PNG/SVG形式

- **現状**: 「今後実装予定です」メッセージ表示
- **将来**: Canvas APIを使用した画像出力を予定

---

4. データ仕様

4.1 Props定義

```
interface PetriNetViewerProps {
  maimlData?: string | null;
  dotData?: string | null;
  enhancedMode?: boolean;
  performanceStats?: unknown;
  propertyDisplayMode?: 'modal' | 'drawer';
  containerElementId?: string;
  selectedMethodId?: string;
  onOpenSubNetViewer?: (subnetData: {
    transitionName: string;
    maimlData: string;
    templateMap: Record<string, { element: Element }>;
    petriNetData: {
      places: Array<{
        id: string;
        data: {
          label?: string;
          stereotype?: string;
          template?: { id: string };
          templateId?: string;
        };
      }>;
      arcs: Array<{
        id: string;
        source: string;
        target: string;
      }>;
    };
  }) => void;
}
```

Props説明:

Prop名	型	必須	説明
maimlData	string | null	-	MAIML XML文字列
dotData	string | null	-	DOT形式データ（現在未使用）
enhancedMode	boolean	-	Enhanced Mode表示フラグ
performanceStats	unknown	-	パフォーマンス統計情報
propertyDisplayMode	'modal' | 'drawer'	-	プロパティ表示モード（デフォルト: 'modal'）
containerElementId	string	-	DrawergetContainer用要素ID
selectedMethodId	string	-	表示するメソッドID（複数メソッド対応）
onOpenSubNetViewer	Function	-	サブネットビューアー開く時のコールバック

4.2 State定義

```
const [nodes, setNodes, onNodesChange] = useNodesState([]);
const [edges, setEdges, onEdgesChange] = useEdgesState([]);
const [loading, setLoading] = useState(false);
const [error, setError] = useState<string | null>(null);
const [showArcLabels, setShowArcLabels] = useState(false);
const [multiMethodMode, setMultiMethodMode] = useState(false);
const [methodName, setMethodName] = useState<string>('');
const [drawerOpen, setDrawerOpen] = useState(false);
const [selectedNode, setSelectedNode] = useState<Node | null>(null);
const [selectedNodeId, setSelectedNodeId] = useState<string | null>(null);
const [structuredProperties, setStructuredProperties] = useState<{
  basic: Record<string, PropertyObject>;
  template: Record<string, PropertyObject>;
  data: Record<string, PropertyObject>;
}>({ basic: {}, template: {}, data: {} });
const [selectedPropertiesByNode, setSelectedPropertiesByNode] = useState<Record<string, {
  basic: string[];
  template: string[];
  data: string[];
}>>({});
const [petriNetStructure, setPetriNetStructure] = useState<...>(...);
const [templateMap, setTemplateMap] = useState<Record<string, { element: Element }>>({});
```

State説明:

State名	型	初期値	説明
nodes	Node[]	[]	React Flowノード配列
edges	Edge[]	[]	React Flowエッジ配列
loading	boolean	false	ロード中フラグ
error	string | null	null	エラーメッセージ
showArcLabels	boolean	false	アーク名表示フラグ
drawerOpen	boolean	false	プロパティDrawer表示フラグ
selectedNode	Node | null	null	選択中のノード
structuredProperties	Object	{...}	構造化プロパティ（Basic/Template/Data）
selectedPropertiesByNode	Object	{}	ノードごとの選択プロパティ
petriNetStructure	Object	null	SubNetViewer用のPetri Net構造
templateMap	Object	{}	SubNetViewer用のテンプレートマップ

4.3 その他の型定義

```
interface PropertyObject {
  key?: string;
  value?: unknown;
  type?: string;
  description?: string;
  defaultValue?: string;
}
```

---

5. API連携

5.1 使用するAPI一覧

#	API名	メソッド	エンドポイント	状態	説明
-	（なし）	-	-	-	API呼び出しは親コンポーネントで実施

備考:

- PetriNetViewerコンポーネント内では直接APIを呼び出さない
- `maimlData` はPropsとして親コンポーネントから受け取る

---

6. エラーハンドリング

6.1 エラーケース一覧

エラーケース	対処方法
MAIMLパースエラー	errorメッセージ設定、Alert表示
メソッドが見つからない	「メソッドが見つかりません」メッセージ表示
変換処理エラー	「ペトリネット構造の読み込みに失敗しました」メッセージ表示

6.2 エラー処理コード

```
try {
  const parser = new MaimlParserTyped(maimlData);
  const methodsData: Method[] = parser.getMethods();
  // ... 処理 ...
```

```
  } catch (err) {
    console.error('❌ [PetriNetViewer] 初期化エラー:', err);
    setError(err instanceof Error ? err.message : 'ペトリネット構造の読み込みに失敗しました');
  } finally {
    setLoading(false);
  }
```

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/viewer/PetriNetViewer.tsx`

7.2 関連ファイル

- **ユーティリティ**:
  - `frontend/src/utils/maimlParserTyped.ts`
  - `frontend/src/utils/petriNetExtractor.ts`
  - `frontend/src/utils/reactFlowConverter.ts`
- **型定義**: `frontend/src/types/maiml-unified.ts`
- **カスタムノード**:
  - `frontend/src/components/nodes/MaterialTemplateNode.tsx`
  - `frontend/src/components/nodes/ConditionTemplateNode.tsx`
  - `frontend/src/components/nodes/ResultTemplateNode.tsx`
  - `frontend/src/components/nodes/TransitionNode.tsx`
- **ダイアログ**:
  - `frontend/src/components/viewer/dialogs/PropertyViewerWithSelection.tsx`

7.3 依存コンポーネント

- `<ReactFlow />` - React Flowメインコンポーネント
- `<MiniMap />` - ミニマップコンポーネント

- `<Controls />` - コントロールパネル
- `<Background />` - 背景グリッド
- `<Drawer />` - Ant Design Drawer
- `<PropertyViewerWithSelection />` - プロパティ選択ビューア

7.4 依存ライブラリ

- `reactflow` : ReactFlow、MiniMap、Controls、Background、useNodesState、useEdgesState、ReactFlowProvider、Panel
- `antd` : Card, Button, Space, Typography, Alert, Spin, Flex, Switch, Dropdown, message, Drawer
- `@ant-design/icons` : DownloadOutlined, ZoomInOutlined, ZoomOutOutlined, ReloadOutlined, FullscreenOutlined

---

8. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を再作成	Cline